LAB 4

Vectorization

from scalar loops to cache-aware matrices

SIMD · intrinsics · SMT · locality · matrix multiplication

One question guides the lab:

What limits the speedup now?

SCALAR



one element at a time

SIMD



one instruction · several lanes

Then ask whether caches and threads let those lanes stay busy.

ROADMAP

The lab builds one idea at a time

1

See

independent loop iterations

2

Verify

compiler report + guided assembly

3

Intervene

intrinsics when analysis fails

4

Stress

threads, working set, stride

5

Combine

SIMD + locality in matmul

Predict → run → inspect → explain. Timing is evidence, not proof.

CONCEPT

SIMD and threads expose different parallelism

SIMD

one instruction stream



Same operation across independent data lanes

AVX2: 4 doubles · AArch64 NEON: 2 doubles

DATA-LEVEL PARALLELISM

THREADS

multiple instruction streams



SMT can hide stalls; it does not duplicate execution units.

THREAD-LEVEL PARALLELISM

SETUP

Record the machine before measuring

The same source runs on Intel/AMD x86-64 and 64-bit Raspberry Pi OS.

CAPTURE WITH EVERY SUBMISSION

```
$ uname -m
$ lscpu -e=CPU,CORE,SOCKET,NODE
$ lscpu | egrep 'Model name|Core|Thread|L1d|L2|L3'
$ grep -m1 -Ei '^(flags|features)' /proc/cpuinfo
```

x86-64

- SSE2 baseline
- AVX2 when available
- SMT may be present

AArch64

- 128-bit NEON
- Pi 3B+ / 4 / 5
- no SMT siblings

Repeated CORE IDs reveal SMT siblings. Adjacent CPU numbers do not.

ISA support tells us what is possible—not what the compiler used.

METHOD

A fair compiler experiment changes one knob

-O3 optimizes · -march selects the ISA · -ftree-* controls vectorizers

1

lab4_scalar

```
-O3 -march=native
-fno-tree-loop-vectorize
-fno-tree-slp-vectorize
```

optimized scalar control

2

lab4_auto

```
-O3 -march=native
-ftree-loop-vectorize
-ftree-slp-vectorize
```

normal auto-vectorization

3

lab4_fast

```
auto flags
+ -ffast-math
(relaxed FP)
```

optional semantics test

PROOF LADDER

report → assembly guide → time

-O0 versus -O3 is not a SIMD experiment.

Stage 1 · Give the compiler an easy loop

AXPY

```
for (size_t i = 0; i < n; ++i)
    a[i] += alpha * b[i];
```

Why GCC can vectorize it

- iterations are independent
- unit-stride arrays
- same operation on every element

one scalar iteration → four AVX2 lanes



CONTROLLED A/B

```
./lab4_scalar axpy 1000000 200
./lab4_auto   axpy 1000000 200
```

Vector width is potential—not a promised speedup. AXPY can become bandwidth-bound.

COMPILER DECISION

Stage 1B · When the compiler needs help

All three paths perform the same independent element-wise addition.

AUTO

```
a[i] += b[i]
```

AUTO-VECTORIZED

→

OPAQUE

```
a[i] = add(a[i], b[i])
```

MISSED

→

MANUAL

```
load · add · store
```

EXPLICIT SIMD

RUN

```
make vec-check
./simd_demo auto       1000000 200
./simd_demo opaque     1000000 200
./simd_demo intrinsics 1000000 200
```

compiler limitation ≠ true dependence

Worked example · we extend the intrinsics

X86-64 · AVX2

```
__m256d va = _mm256_loadu_pd(a+i);
__m256d vb = _mm256_loadu_pd(b+i);
_mm256_storeu_pd(a+i,
    _mm256_add_pd(va, vb));
i += 4;
```

same algorithm →

AARCH64 · NEON

```
float64x2_t va = vld1q_f64(a+i);
float64x2_t vb = vld1q_f64(b+i);
vst1q_f64(a+i,
vaddq_f64(va, vb));
i += 2;
```

In simd_intrinsics_demo.c, change every path to a[i] += 2.0 × b[i].

Do not forget the scalar tail.

LEGALITY

Stage 2 · A real dependence serializes the loop

RECURRENCE

```
for (size_t i = 1; i < n; ++i)
    a[i] = 0.5 * a[i - 1] + b[i];
```

Iteration i consumes the value produced by iteration i−1.



must preserve order

NEGATIVE CONTROL

```
./lab4_scalar dependency 1000000 20
./lab4_auto   dependency 1000000 20
```

Intrinsics cannot simply remove a true data dependence.

Stage 3 · More arithmetic makes SIMD visible

AXPY few operations  **loads + stores dominate**

POLY many operations  **vector units own more time**

COMPARE

```
taskset -c A ./lab4_scalar poly 1 2000000 100
taskset -c A ./lab4_auto   poly 1 2000000 100
```

Prediction: SIMD gain for POLY > SIMD gain for AXPY

Keep total work fixed when adding a thread

x86 with SMT

thread 0

thread 1

→ same physical core
shared vector units

SMT

Raspberry Pi

core 0

core 1

no SMT · compare physical cores

FIXED TOTAL WORK

```
taskset -c A   ./lab4_auto poly 1 ...
taskset -c A,B ./lab4_auto poly 2 ...
```

PAPER INSIGHT

Vectorization and cache reuse can disagree

VECTORIZE

more lanes

split loops

↔

winner depends on working set

↔

REUSE

fewer transfers

fuse loops

> **Measure the crossover. Do not memorize one form as “optimized.”**

Shen, Chabbi & Liu · PMAM 2018

LOCALITY VS SIMD

Stage 4 · Same result, different resource tradeoff

FUSED · ONE PASS

```
for (i = 1; i < n; ++i) {
  A[i] = 3.0 * B[i-1];
  B[i] = 0.5 * B[i] + C[i];
}
```

IMMEDIATE REUSE

but a memory dependence blocks ordinary SIMD

SPLIT · TWO PASSES

```
for (i = 1; i < n; ++i)
  B[i] = 0.5 * B[i] + C[i];
for (i = 1; i < n; ++i)
  A[i] = 3.0 * B[i-1];
```

BOTH VECTORIZE

but B is revisited after a whole-array pass

Predict twice: which wins when the arrays fit in cache? Which wins beyond LLC?

EXPERIMENT

Find the crossover instead of choosing a winner

ENDPOINTS

```
# small working set
./lab4_auto tradeoff {fused,split} 4096 20000

# beyond last-level cache
./lab4_auto tradeoff {fused,split} 8000000 10
```

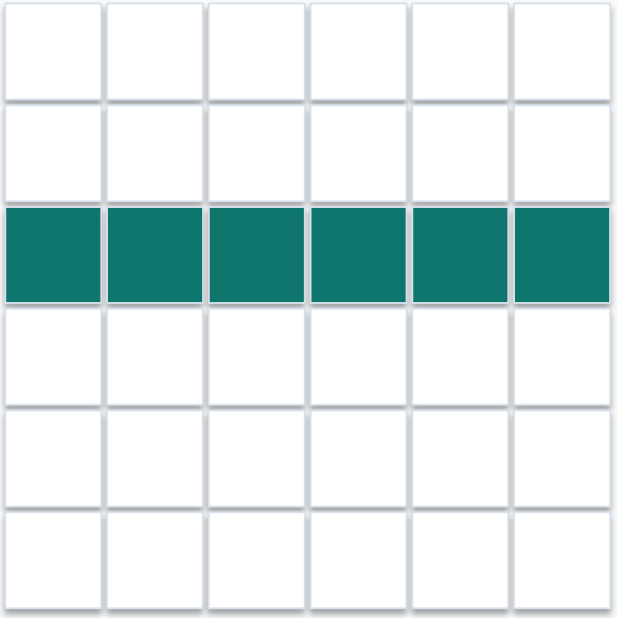
HYPOTHESIS—NOT ANSWER



Repeat with lab4_scalar to separate SIMD gain from locality cost.

Stage 5 · Traversal order changes the memory stream

ROW ORDER

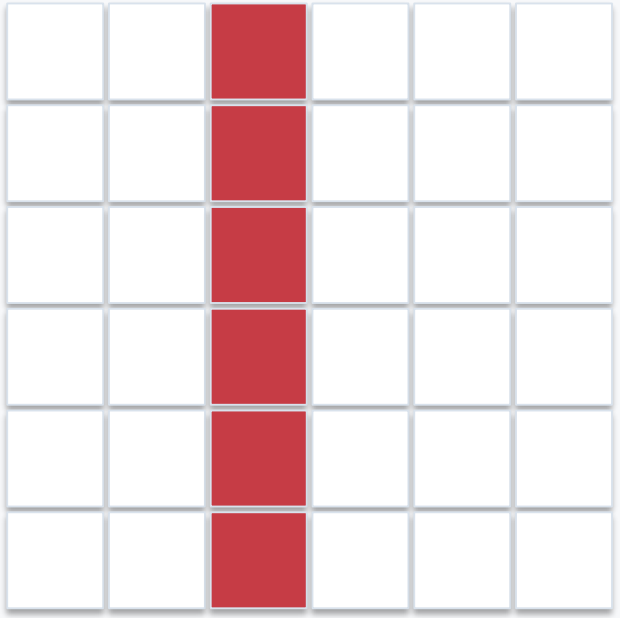


→ unit stride

CACHE-FRIENDLY

VECTORIZABLE

COLUMN ORDER



stride = N × 8 B

WASTED CACHE LINES

SIMD UNLIKELY

EXPERIMENTAL DESIGN

A 2×2 experiment separates locality from SIMD

	ROW	COLUMN
SCALAR	**locality only** scalar · row	**neither advantage** scalar · column
AUTO	**locality + SIMD** auto · row	**strided; SIMD unlikely** auto · column

```
./lab4_{scalar,auto} matrix {row,col} 2048 4
```

Stage 6 · ijk matrix multiplication fights the layout

IJK

```
for (i)
  for (j)
    for (k)
      C[i][j] += A[i][k] * B[k][j];
```

INNER k LOOP

A	row access	contiguous
B	column access	stride N × 8 B
C	one scalar	reduction dependency

Same ≈$2N^3$ useful flops; very different data movement.

Loop interchange repairs the inner loop

IKJ

```
for (i)
  for (k) {
    double aik = A[i][k];
    for (j)
      C[i][j] += aik * B[k][j];
  }
```

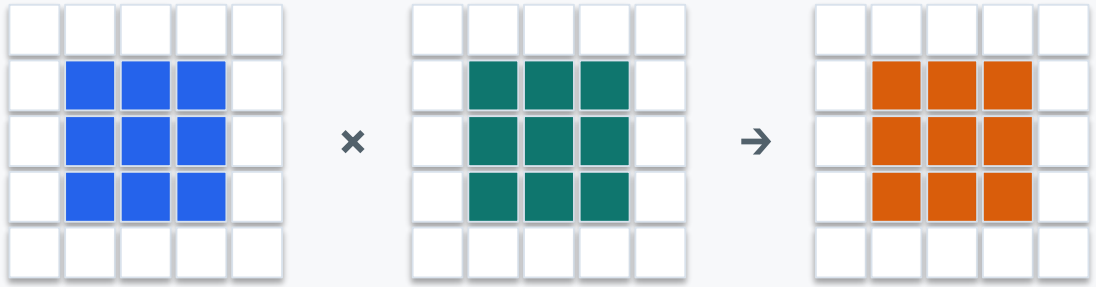
INNER j LOOP

- B row is contiguous
- C row is contiguous
- A[i][k] is reused from a register
- independent j iterations can vectorize

MEASURE

```
./lab4_auto mm ijk 512 64
./lab4_auto mm ikj 512 64
```

Blocking keeps reuse close to the core



MM_BLOCKED() · SUPPLIED REFERENCE

```
for (ii, kk, jj in tiles)
  for (i in ii tile)
    for (k in kk tile)
      for (j in jj tile)   // unit stride
```

Tune—do not guess

```
for b in 16 32 48 64 96 128; do
  ./lab4_auto mm blocked 512 $b
done
```

Too small: overhead.
Too large: capacity/conflict pressure.

HANDS-ON

Challenge · write mm_blocked_intrinsics()

1. **BASELINE** — time blocked + scalar starter
2. **BROADCAST** — copy aik into SIMD lanes
3. **COMPUTE** — load B/C · mul · add · store
4. **TAIL** — retain scalar cleanup
5. **COMPARE** — assembly + median time

AUTO vs MANUAL

blocked

compiler-generated SIMD

intrinsics

our AVX2 / SSE2 / NEON

same N · same block · same work

Edit only SIMD TODOs; keep mm_blocked() unchanged as the control.

PROTOCOL

Measurement guardrails

01	CALIBRATE	fastest run ≥ ~0.1 s
02	REPEAT	at least 3; report median
03	VERIFY	matching checksums / PASS
04	PLACE	record CPU IDs + topology
05	INSPECT	report + guide + time
06	CONTROL	same work and -O3 baseline

perf counts setup too; elapsed_s covers the timed kernel region.

PERF STAT

Counters explain the time—not replace it

REPEAT THREE TIMES

```
make perf-stat PERF_CPU=A \
  PERF_COMMAND='./lab4_auto mm blocked 512 64'

# same algorithm · handwritten SIMD
make perf-stat PERF_CPU=A \
  PERF_COMMAND='./lab4_auto mm intrinsics 512 64'
```

CYCLES

Did it finish with less processor work?

INSTRUCTIONS

SIMD counts once while doing several lanes

IPC

Useful context—not a standalone speed score

CACHE

Use PMU-specific L1/L2/LLC events when available

Same command · same input · same CPU. Report unsupported or multiplexed events.

DELIVERABLE

What to submit

1 **SIMD / THREAD TABLE**

times, speedups, placement

2 **AUTO / OPAQUE / MANUAL**

verdict + matching checksum

3 **PROOF**

guide or compiler-report excerpt

4 **CROSSOVER**

working set vs fused/split

5 **MATRIX EVIDENCE**

auto vs intrinsics + block sweep

6 **COUNTER EVIDENCE**

cycles · instructions · IPC · cache

7 **EXPLANATIONS**

claim → evidence → mechanism → next test

A surprising result with a careful next test beats a forced expected result.

SYNTHESIS

The five questions to ask of every hot loop

1

DEPENDENCE

Can iterations run independently?

→

2

ACCESS

Are lane accesses contiguous?

→

3

CODEGEN

Did the compiler emit SIMD?

→

4

REUSE

Does the working set stay nearby?

→

5

THREADS

Will more cores add throughput?

Fast matrix multiplication is the result of the whole sequence—not one trick.